Models as Parametric Kernels

An Order, an Operator and a Polynomial:

Deriving the Facts a Model Governance System Otherwise Declares

Ashutosh Sinha
Independent Researcher
ajsinha@gmail.com

Abstract

A large regulated institution operates thousands of quantitative artefacts — closed-form pricers, market-calibrated surfaces, estimated scorecards, learned classifiers, online-adaptive systems, foundation-model applications, vendor black boxes, elicited rating schemes, deterministic rule sets — and must govern them as one population. The systems built to discharge that obligation run on *declared* facts: somebody types what kind of artefact this is, somebody asserts that this version may replace that one, somebody writes down what a training set could have known, somebody records in prose what a conclusion rests on. Declared facts decay silently, and every pathology practitioners report is a declaration that has drifted from the thing it describes.

We argue that four of these facts need not be declared at all, and give the algebra that derives each. A model is a morphism in **Para**(**Stoch**): a parameter object P and a kernel $f\colon P \otimes X \to \mathcal{D}(Y)$. From *how P is inhabited* we derive the trainability class, so that "some governed artefacts are never trained" becomes a type-level fact rather than a special case. From a single partial order on schemas — $A \sqsubseteq B$, read *A can stand in for B* — we derive substitutability, featureset adequacy, and whether two artefacts may be wired together; the order is a lattice, its meet is partial and informatively so, and one relation replaces four implementations that were free to disagree. From the point-in-time read presented as an operator $\mathrm{AsOf}(R, \ell, a)$ we derive reproducibility: the operator is idempotent, commutes with projection, monotone in the observation time and — because the ingest bound is $\min(\ell, a)$ — *saturating* at the label, which is the guarantee that a training row re-assembled a year later is the row that was assembled originally. From the free commutative semiring $\mathbb{N}[X]$ we derive every question about what a claim or a feature rests on, each as a pushforward along a homomorphism; the ingest clock of a derived feature is one of them, which upgrades a rule that could be forgotten into a homomorphism that has no exceptions.

We are equally explicit about what does not derive. Aggregate risk cannot be compositional, and the obstruction is exactly the comonoid copy map. The currency valuation $(\max, \max)$ is not a semiring — its zero does not annihilate, and no choice of constants repairs it — so the universal property does not reach it. And some facts *constitute* the standard they would be checked against: a tiering rule, a regime encoding, an approval. That boundary, drawn for governance reasons, turns out to be the same boundary that decides where machine generation is admissible, which we regard as the most useful consequence of the account.

Each claim is marked with where it is enforced. Eighteen of the twenty-one stated laws execute against generated inputs; five do not, and are named with the reason in the place a reader would look for the executable one. That marking is itself part of the argument: a law that is stated and not executed prevents nothing, and a document implying otherwise is the failure the discipline exists to prevent.

1 Introduction

1.1 The domain

Consider the population of quantitative artefacts operated by a large financial institution. It includes a Black–Scholes implementation whose parameters come from financial theory and are never estimated from data; a stochastic volatility surface re-solved against market quotes every morning; a logistic scorecard refitted annually on a historical sample; a gradient-boosted classifier retrained weekly on transaction streams; an adaptive threshold that updates itself continuously in production; a retrieval-augmented language model drafting regulatory narratives; a third-party credit score whose internals are contractually unavailable; a country-risk rating scheme whose weights are set by a committee; and several thousand rule sets and spreadsheets.

These artefacts have essentially nothing in common at the level of implementation. Yet they are subject to a single normative obligation. Supervisory guidance requires an institution to maintain an inventory of them, classify each by materiality, subject each to independent challenge proportionate to that classification, monitor performance, document design and limitations, control change, and assess risk both individually and *in aggregate*, accounting for "interactions and dependencies among models; reliance on common assumptions, data, or methodologies" [35, 36]. Different jurisdictions define the governed population differently: one regime narrows it to complex quantitative methods applying statistical, economic or financial theory, explicitly excluding deterministic rules and, remarkably, generative systems; another defines it broadly enough to include expert-judgment inputs and qualitative outputs [36, 37]. An institution operating in both must satisfy a narrow and a broad scope simultaneously over one population.

1.2 Declared facts, and why they rot

The systems built for this task fail in recognisable ways. It is usual to present those failures as four pathologies. We prefer a sharper diagnosis, because it points at the remedy: in each case a *fact that could have been derived from structure was instead declared by a person*, and a declared fact is only true at the moment it is typed.

D1: kind is declared. A field records that this artefact is "an ML model" or "a pricer" or "other". Downstream, evidence requirements switch on that field, so a closed-form pricer is asked for its training dataset. That is not a missing value; it is a type error, and a register that cannot say so accumulates records that mean nothing. The declaration is also the only thing standing between the taxonomy and reality: nothing checks that an artefact declared "rule set" has no fitted parameters.

D2: fit is declared. "This version may replace that one." "This featureset provides what that model reads." "This model's output goes into that model." Each is a claim that one thing can stand where another stood. In the systems we have examined each is answered by its own code path, and four implementations of one relation eventually disagree — predictably in the direction of permitting more, because that is the direction in which nobody files a bug.

D3: currency is declared. The condition under which a training row is admissible is written into a query by whoever wrote the query. Whether the query implements the condition is not checked, and the failure — a fact used before it was knowable — is invisible to every performance metric, because it improves them.

D4: support is declared. An evidence log records, in prose, that a version was validated. Nothing in the record establishes what the conclusion rests on, so nothing can compute whether a cited set suffices, what the cheapest route to closing a gap is, or whether a document is stale. The claim is unfalsifiable, which is the opposite of what assurance requires.

1.3 The thesis

Each of D1–D4 is a fact about structure that has been recorded as a fact about opinion. Our thesis is that all four derive, and that the algebra required is small: a definition, an order, an operator and a polynomial.

1. Section 2 defines a model as a morphism of **Para**(**Stoch**) and derives the trainability class from how the parameter object is inhabited (Proposition 2.3). D1 dissolves: training is one way of inhabiting P, not part of being a model.
2. Section 3 gives *one* partial order on schemas, $A \sqsubseteq B$ read as "A can stand in for B", shows it is a lattice with partial meet and empty top (Theorem 3.6 and proposition 3.8), and shows that the four questions of D2 are that one comparison (Proposition 3.9).
3. Section 4 makes model-to-model composition a type-check rather than a drawing (Proposition 4.1), and then proves the negative result that bounds it: no aggregate risk assignment is both compositional and sensitive to shared dependency, with the comonoid copy map as the exact obstruction (Theorem 4.6).
4. Section 5 presents the point-in-time read as an operator and proves four properties of it, of which the fourth — saturation at the label — *is* reproducibility (Proposition 5.2 and corollary 5.4). D3 dissolves: the condition is a property of an operator, checked by property tests and by an independent recomputation, rather than a convention in a query.
5. Section 6 annotates evidence and derived features over the free commutative semiring $\mathbb{N}[X]$ and shows that every other question is a pushforward along a homomorphism (Proposition 6.1), including the ingest clock of a derived feature (Proposition 6.3). D4 dissolves. We also prove that the currency valuation cannot be one of these homomorphisms (Proposition 6.4).
6. Section 2.4 indexes evidence requirements as the fibres of a fibration, and shows that the base of that fibration must itself be derived: a declared base cannot deliver conservative extension and totality together (Proposition 2.7). Section 2.5 treats the one class whose parameter object has logical rather than numerical structure, and states exactly what becomes decidable there and what does not (Proposition 2.8).
7. Section 7 draws the complement honestly: the facts that must be declared because they *constitute* the standard against which anything else is checked.
8. Section 8 states the discipline that keeps the rest from becoming ornament, gives the register of twenty-one laws with the three that do not execute named, and states the properties an executable law delivers that a documented one does not.
9. Section 9 observes that the derive/declare boundary of Sections 3 and 5 to 7 is exactly the boundary that decides where machine-generated output may be accepted (Propositions 6.5 and 9.2).

1.4 What is borrowed and what is new

We are explicit, because the value of an integrative paper depends on it. Markov categories are due to Fritz [1] and, in the string-diagrammatic form we use, to Cho and Jacobs [2]; the

underlying monad is Giry's [3]. The **Para** construction is due to Fong, Spivak and Tuyeras [4], elaborated by Cruttwell, Gavranović, Ghani, Wilson and Zanasi [5] and by Capucci and collaborators [6]. Lattice theory is standard [7, 8], as is the variance discipline our order recovers [9, 10]. Provenance semirings and the universality of $\mathbb{N}[X]$ are due to Green, Karvounarakis and Tannen [11, 12], with the semiring hierarchy of [13, 14]. Bitemporal data management is Snodgrass's [16], and reached the standard in SQL:2011 [17]. Assume–guarantee contracts are due to Benveniste and collaborators [22]; institutions to Goguen and Burstall [20]; sound abstraction to Cousot and Cousot [24]; well-behaved lenses to Foster and collaborators [25]; property-based testing to Claessen and Hughes [26]. None of these is our contribution.

What we claim is new, in decreasing order of confidence: (i) the derivation of the trainability classification from the inhabitation of P (Proposition 2.3), and the observation that it makes "never trained" a type and not an exception; (ii) the identification of *one* schema order as the common content of substitutability, featureset adequacy and composability, together with the informative partiality of its meet (Propositions 3.8 and 3.9); (iii) the presentation of the point-in-time read as an operator and the saturation law as the reproducibility guarantee (Proposition 5.2 and corollary 5.4) — the point-in-time *condition* is folklore, and what appears not to be stated anywhere is that the correct ingest bound is $\min(\ell, a)$ and that this is what makes the read stable; (iv) Theorem 4.6 and its diagnosis of the copy map as the obstruction; (v) the application of semiring provenance to assurance evidence and, one layer across, to derived features, so that a data-engineering rule becomes a homomorphism (Proposition 6.3); (vi) Proposition 6.4, an elementary negative result that we record because the corresponding positive claim appears in our own earlier design notes and, we suspect, in others'; (vii) Proposition 2.7, which we believe to be new and which is the paper's own thesis applied to its own machinery: an indexed family of governance obligations cannot be both freely extensible and provably total unless its index is computed rather than recorded; (viii) the observation that the derive/declare boundary and the oracle boundary coincide (Section 9).

1.5 How to read this paper, and how to check it

The mathematics is elementary; the vocabulary is not evenly distributed across the audiences this work addresses. Every formal statement is therefore followed by a shaded *plain terms* gloss, and where it earns one, a worked example from banking. A reader who wants the argument without the notation can follow the shaded material alone and lose nothing essential.

Every formal claim carries a marker saying where it is enforced in the reference implementation. The marker names a test, a module, or — for five of the twenty-one laws — says plainly that the claim is stated and not executed. We regard the second kind of marker as the more important one, for the reason Section 8 gives: a document implying that every law is checked, while six are not, is the exact failure that section exists to prevent.

2 The kernel, and what the parameter object decides

2.1 Ambient setting

We work in a Markov category: a symmetric monoidal category $(\mathcal{C}, \otimes, I)$ in which the unit I is terminal and every object X carries a commutative comonoid structure $\mathrm{copy}_X \colon X \to X \otimes X$, $\mathrm{del}_X \colon X \to I$, compatible with the monoidal product and with del natural [1, 2]. Morphisms behave as Markov kernels; the canonical example is **Stoch**, the Kleisli category of the Giry monad on measurable spaces [3]. A morphism f is *deterministic* when $\mathrm{copy}_Y \circ f = (f \otimes f) \circ \mathrm{copy}_X$.

Two structural features earn their place immediately. Terminality of I says every morphism may be discarded — outputs need not be used, which is what makes an artefact's *exposure* a meaningful separate quantity. Explicit copying says an output may be consumed by several

downstream artefacts, which, as Section 4 shows, is exactly the structure responsible for non-compositional aggregate risk. A Cartesian category has both properties implicitly and therefore cannot make either visible.

2.2 Definition

Definition 2.1 (Model)**.** A *model* is a morphism in $\mathbf{Para}(\mathcal{C})$ for a Markov category $\mathcal{C}$: a pair (P, f) of a *parameter object* $P \in \mathcal{C}$ and a kernel

$$f\colon P \otimes X \longrightarrow Y,$$

with X the *input object* and Y the *output object*. Composition of $(P, f)\colon X \to Y$ with $(Q, g)\colon Y \to Z$ is $(Q \otimes P,\ g \circ (\mathrm{id}_Q \otimes f))$.

Runs: `core/domain/algebra.py::ParametricKernel`

> **In plain terms.** A model is a machine with *dials*. X is what you feed in, Y is what comes out, and P is the dials that decide how the machine behaves. The definition insists on keeping the dials separate from the machinery, because that separation is where all the leverage is.
> "Kernel" means the output may be random: feed in the same input twice and you may get different answers. That covers a language model sampling text. Deterministic behaviour is the special case where the randomness is trivial.
> The composition rule says: plug one machine into another and the combined machine's dials are *both* sets of dials together. Nothing is absorbed.

Definition 2.2 (Fitting morphism)**.** A *fitting morphism* for (P, f) is a morphism $\varphi\colon D \to P$, where D is an object of *fitting evidence*. A model is *accompanied* when φ is available to the governing party and *opaque* otherwise.

Definition 2.2 is where the account departs from received practice. Estimation, calibration, training, elicitation, configuration and authorship are not different *kinds of model*; they are different fitting morphisms into the parameter object of one kind of thing.

> **Worked example: two artefacts that look nothing alike**
> A **Black–Scholes pricer** takes spot, strike, tenor, rate and volatility and returns a price. Its formula comes from theory. There are no dials: P is terminal. Feed it the same inputs and it returns the same price forever.
> A **SABR volatility surface** takes a strike and a tenor and returns an implied volatility — but first, four parameters must be solved against this morning's quotes. Those four numbers are P, and they are different every day.
> Same shape, entirely different operational reality, and the difference lives in P.

2.3 Trainability is derived

Proposition 2.3 (Classification)**.** *Let $\kappa(P)$ record whether P is terminal and whether it is accessible, let $\varphi \in \{\textsc{none}, \textsc{calibrate}, \textsc{estimate}, \textsc{train}, \textsc{configure}, \textsc{elicit}, \textsc{author}\}$ name the fitting morphism, and let $\mathrm{ad} \in \mathbb{B}$ record whether the artefact updates P in production. Then the informal taxonomy of governed artefact kinds is the value of a total function of $(\kappa, \varphi, \mathrm{ad})$, computed in this order:*

$$\begin{array}{ll}
P \text{ inaccessible} & \mapsto\ \mathsf{T6} \\
P \cong I \text{ terminal} & \mapsto\ \mathsf{T0} \\
\varphi = \textsc{train} \wedge \mathrm{ad} & \mapsto\ \mathsf{T4} \\
\text{otherwise, by } \varphi: & \textsc{calibrate} \mapsto \mathsf{T1},\quad \textsc{estimate} \mapsto \mathsf{T2},\quad \textsc{train} \mapsto \mathsf{T3}, \\
 & \textsc{configure} \mapsto \mathsf{T5},\quad \textsc{elicit} \mapsto \mathsf{T7},\quad \textsc{author} \mapsto \mathsf{T8}.
\end{array}$$

The classification is never stored and never declared.

Runs: `core/domain/algebra.py::ParametricKernel.trainability_class`

Proof. The function is total because the cases are checked in order and the final dispatch has a default. The two extremal cases are the mathematically interesting ones. T0 is $P \cong I$: since I is terminal, $I \otimes X \cong X$, the model is a bare kernel $X \to Y$, and the fitting problem is vacuous — so a demand for fitting evidence is not an unanswered question but an ill-typed one. T6 is the case in which P exists but neither P nor φ is available, so the only accessible datum is the composite $f(p, -)$ for the vendor's fixed p, which is an ordinary morphism $X \to Y$; that is precisely why behavioural evidence is the only evidence obtainable. Exhaustiveness of the remaining rows over current practice is an empirical claim about the domain, not a mathematical one, and we make it as such. □

Class	Parameter object P	Fitting morphism φ	Refresh trigger
T0 Analytical	terminal	none; nothing to fit	specification or regulation
T1 Calibrated	calibration set	a solver on quoted instruments	each market close
T2 Estimated	coefficients	an estimator on a sample	a periodic refit
T3 Learned	weights	a training run	schedule, drift, decay
T4 Adaptive	weights that move in production	a training run, then itself	continuously, by the model
T5 Configured	base model, prompt, corpus, tools	configuration and retrieval	any of those, or the base model
T6 Opaque	exists, unreachable	somebody else's, unavailable	a vendor release
T7 Elicited	weights a panel agreed	an elicitation protocol	the committee cycle
T8 Authored	a rule set	authorship	a policy change

In plain terms. Practitioners argue endlessly about whether a pricer "is a model", whether a rule set counts, whether a chatbot belongs in the inventory. The table says the argument is misconceived. These are not nine kinds of thing. They are one kind of thing, distinguished by *how the dials get set* — and because that is computed rather than typed in, nobody can get it wrong on a form.
The two ends are worth dwelling on. At the top the dials do not exist, so asking about them is meaningless. At the bottom of the opaque row the dials exist but are behind a contract, so asking about them is futile. Everything else is in between.

Corollary 2.4 (Fitting evidence is typed)**.** *A demand for fitting evidence is well-typed exactly when the class is not T0 or T6. In the first case there is no P to inhabit; in the second there is no accessible φ to evidence.*

Runs: `core/domain/algebra.py::ParametricKernel.requires_fitting_evidence`

Worked example: three artefacts through the same lens

An SA-CCR exposure calculator. The formula is prescribed by regulation; P is terminal. There is no dataset, no estimator, no retraining schedule. What must be evidenced is that the implementation computes what the regulation says — a test suite against reference values, not a validation sample. Any system demanding a training set here asks a meaningless question and gets a meaningless answer typed into a mandatory field.
A small-business PD scorecard. P is forty-two coefficients; φ is a logistic regression over eight years of originations. Evidence: the sample, its representativeness, the estimator, out-of-time performance, calibration.
A bought-in bureau score. P exists — somebody at the bureau fitted it — but neither P nor φ is available. Demanding development documentation is futile. What you can do, and what the structure says is the *only* thing you can do, is watch the composite: compare its outputs against your own realised defaults. That is not a compromise forced by commercial reality; it is what the

> mathematics permits.

2.4 The classification is the index

Almost every structure in this domain is a family indexed by something else: versions by artefact, calibrations by version, deployments by environment, and — the one that matters here — *evidence requirements by kind of artefact.* The mathematics of indexed families is the fibration, and the governance question it answers is what a new kind of artefact costs.

Let $p\colon \mathcal{E} \to \mathcal{B}$ be a fibration with $\mathcal{B}$ a category of artefact kinds, and let the fibre $\mathcal{E}_c$ over kind c carry the four things that vary with kind: an evidence schema, a lifecycle, a set of monitors that can answer a question about it, and a set of documents that compile for it.

Proposition 2.5 (Conservative extension)**.** *Let $\mathcal{B}^+$ extend $\mathcal{B}$ by a new object c with a chosen fibre $\mathcal{E}_c$ and no new morphisms into or out of existing objects. Then the inclusion $\mathcal{E} \hookrightarrow \mathcal{E}^+$ is full and faithful, commutes with the projections, and satisfies $\mathcal{E}^+_b = \mathcal{E}_b$ for every $b \in \mathcal{B}$. No judgement expressible over $\mathcal{B}$ is altered by the extension.*

Proof. $\mathcal{E}^+$ is the coproduct $\mathcal{E} + \mathcal{E}_c$ over $\mathcal{B} + \{c\}$; fullness, faithfulness and fibrewise identity are immediate, and the cartesian liftings over $\mathcal{B}$ are unchanged since no morphisms were added. □

Proposition 2.6 (Totality)**.** *The extension claim is worth exactly the totality of the fibration: if some fibre is empty in one of the four facets, there is a kind about which the system has nothing to say, and it will say nothing by omission rather than by refusal. Totality is therefore a checkable property — no fibre is empty — and it is checked before the system serves rather than when a caller relies on it.*

Runs: `core/fibres/registry.py::verify (start-up gate) and ::register (refuses a partial fibre, so the registry cannot hold one); tests/test_laws.py::TestL15NoFibreIsEmpty`

> **In plain terms.** Think of a filing cabinet where each drawer holds one kind of artefact and comes with its own forms: what must be filled in, what gets monitored, what documents come out. The bad design writes the list of drawers into the cabinet's frame, so adding a drawer means rebuilding the cabinet. The good design makes drawers independent, so adding one is adding one — and Proposition 2.5 is the guarantee that adding one cannot disturb what is in the others.
> Proposition 2.6 is the other half, and it is the half that is usually missing: every drawer has to actually contain its forms. A drawer with no forms in it is worse than an absent drawer, because the cabinet looks complete.

The base has to be derived

There is a constraint on $\mathcal{B}$ that the fibration literature has no reason to state and that this domain does. It is the paper's own thesis applied to the paper's own machinery, and it is sharp enough to be a dilemma.

Proposition 2.7 (A declared base admits no total fibration)**.** *Suppose the base $\mathcal{B}$ is a free-text attribute recorded by a person. Then totality (Proposition 2.6) and conservative extension (Proposition 2.5) cannot both be delivered:*

1. *If the admissible values are closed, then extension requires enlarging the closed set, which is a change to the base rather than the supply of a fibre — and Proposition 2.5 no longer describes what adding a kind costs.*
2. *If they are open, then a value with no fibre is producible at any time by writing one down, so the totality check quantifies over a set that does not contain the values actually in use. The gate passes and the fibration is not total.*

Both horns are avoided exactly when the base is derived*: an index computed from the artefact cannot be typed wrong, cannot be extended by accident, and cannot disagree with the artefact it indexes.*

Proof. The two cases are exhaustive for an attribute whose values are supplied rather than computed. In the closed case, admitting a new kind changes $\mathcal{B}$'s object set by an act external to supplying $\mathcal{E}_c$, which is what Proposition 2.5 assumes is the whole of the extension. In the open case, let b be a value no fibre covers; the totality check ranges over the registered indices, which by construction excludes b, so it returns success on a fibration that has no fibre over b. If instead the index is a function of the artefact into a fixed finite set — as κ, φ and ad are in Proposition 2.3 — neither case arises: the index set is fixed, so totality quantifies over all of it, and extension is the supply of a fibre. □

The trainability classification of Proposition 2.3 is such a base. That is the reason it is worth deriving twice over: once so that a closed-form pricer is a type rather than an exception, and once so that the structure indexed by it can be required to be complete. An organisational label — what desk owns this, what business it serves — remains useful for grouping and reporting, and indexes nothing.

> **Worked example: a monitor that ran for two years and meant nothing**
> A discount-curve pricer is registered and somebody attaches a *performance* monitor: discrimination against realised outcomes, evaluated monthly. It runs. It never fires. On the estate screen the model is green and monitored.
> The pricer has no parameters and no fitted relationship to outcomes, so discrimination is not a question about it. The monitor was not failing to detect anything; there was nothing of that kind to detect. And this is worse than an absent monitor, because an absent monitor appears in the coverage worklist and a meaningless one does not — it reads as coverage.
> With a fibre over the class, the monitor kind is checked against what the class can answer, and the attachment is refused with what *is* answerable for a `T0` artefact named instead: whether the inputs are still inside the range it was benchmarked over. The refusal is not a restriction on what validators may do. It is the fibre being read rather than merely held.

2.5 When the parameter object can be inspected

For most classes P is a vector of numbers and the interesting questions about it are statistical. For `T8` it is an authored rule set, and that changes what is askable: a rule set has *logical* structure, so questions about it can be decided rather than estimated.

The condition for this is a restriction, and the restriction is the point. If rules are written in a general expression language, questions about them are questions about programs. Confine a condition to comparisons on declared fields combined by AND, OR and NOT — no arithmetic, no function calls, no free text — and each condition has a disjunctive normal form whose conjunctions reduce to a domain per field: an interval, a permitted set, an excluded set and a null state. Emptiness and containment are then arithmetic.

Proposition 2.8 (Reachability under first-match-wins)**.** *Let a rule set be an ordered list of rules with a required* otherwise *clause, evaluated first match wins. Under the restriction above, "rule r_j is shadowed by some single earlier rule r_i" is decidable, by testing whether r_i's condition covers r_j's. The decision procedure is* sound*: a rule it reports as shadowed is unreachable. It is* incomplete*: a rule covered only by the* union *of two or more earlier rules is not reported.*

Runs: `core/rules/domains.py::covers, satisfiable; core/rules/ruleset.py; core/rules/editor.py`

Proof. Soundness: if r_i's condition covers r_j's, then every input reaching r_j in evaluation order has already matched r_i, so r_j never fires. Incompleteness is exhibited by $r_1 = (x > c)$,

$r_2 = (x \leq c)$ and any r_3: no single earlier condition covers r_3, and their union covers everything. □

We state the limit exactly rather than describing the check as "unreachable rule detection", because the gap is the interesting part. Complete coverage checking is satisfiability over the theory, decidable here but requiring a solver; the promise made is the one delivered — *no rule is shadowed by any single earlier rule* — and a check that claims more than it delivers is precisely the failure mode the analysis exists to find.

> **In plain terms.** Rule sets are read top to bottom and the first match wins. That makes them easy to read and quietly easy to get wrong: a rule that an earlier rule already covers can never fire, and nothing about reading the document tells you so. Somebody believes that rule is in force. It appears in the model card, gets cited in a committee paper, and survives every review — because a rule that never fires also never produces a wrong answer.
> The analysis finds those, and it is deliberately honest about its reach: it catches a rule blocked by one earlier rule, not a rule blocked by two of them working together. Saying which is the difference between a check people trust and a check people switch off.

Remark 2.9 (Why this belongs in this paper). Two of the account's threads meet here. Proposition 2.3 says T8 is a class because of *how* P is inhabited — somebody wrote it down — and Proposition 2.8 says that when P is inhabited that way, and is held in a structured form rather than as text, properties of P become derivable facts about the model. An authored rule set is the most declared parameter object there is, and it is the one whose contents the platform can say the most about.

2.6 What the derivation still reads

Honesty requires the converse. Proposition 2.3 derives the class from $(\kappa, \varphi, \mathrm{ad})$, and of those three, κ is a property of the artefact while φ and ad are recorded by a person. The derivation therefore *shrinks* the declared surface — from a nine-valued taxonomy with nine ways to be wrong, to two facts about the act that produced the parameters — rather than eliminating it. Two consequences deserve statement.

First, the shape of P does not decide the class; the act does. A kernel whose parameters are recorded as learned weights but whose fitting morphism is an estimator is T2, and correctly so: twenty coefficients from a solver are an estimation whatever the storage is called. Nothing cross-checks the two declarations against each other, and a disagreement between them is not currently a finding.

Second, the fallback direction is permissive. Where φ is unrecorded and P is non-terminal, the dispatch of Proposition 2.3 returns T0, and by Corollary 2.4 the artefact is then exempt from fitting evidence. An artefact whose parameters exist but whose provenance nobody recorded is thereby treated like a closed-form pricer. This is the same failure direction D2 names, one level in, and it is the kind of defect the discipline of Section 8 exists to surface.

2.7 Points of P, and what a version is

Proposition 2.10 (Refitting preserves the morphism)**.** *Let (P, f) be a model and $p, p' \in P$ two inhabitants obtained by the same fitting morphism $\varphi\colon D \to P$ from data d, d'. Then (P, f) is unchanged: p and p' are points of the parameter object, not distinct morphisms. Consequently the pair $\big((P, f), p\big)$, not (P, f) alone, determines observable behaviour.*

Proof. Immediate: φ has codomain P, and f is a fixed morphism independent of which point of P is supplied. □

Mathematically trivial; consequential in practice. Standard practice mints a new *version* on every refit, conflating a change of morphism (new φ, P, X or Y) with a change of point (same everything, new data). A supervisor asking whether a model changed between two dates then asks a question with two answers. Separating them makes an inhabitant of P a first-class record with its own identity, immutability and provenance — which φ produced it, from which d, over which window — so that a daily recalibration governs the *procedure* once while a periodic re-estimation governs *each* inhabitant, and both are the same construction at different frequencies.

Proposition 2.11 (Parameter accumulation)**.** *For composable $(P, f)\colon X \to Y$ and $(Q, g)\colon Y \to Z$, the composite has parameter object $Q \otimes P$. In any finite network the composite parameter object is the tensor of the parameter objects of all constituents reachable from the inputs.*

Proof. Immediate from composition in $\mathbf{Para}(\mathcal{C})$ and induction on network size. □

Corollary 2.12 (Change closure)**.** *Any change to the inhabitant of a constituent parameter object changes the inhabitant of the composite parameter object. Recalibration of an upstream artefact is therefore, formally, a change event for every downstream artefact.*

> **Worked example: the daily recalibration nobody calls a change**
>
> At 6:30 each morning a discount curve is re-solved against market instruments; its parameter set changes. Downstream sit a swaption pricer, an XVA engine, a VaR engine, a capital calculator and a capital plan. By Proposition 2.11 every one of them has had its parameter object change. Formally each has changed; operationally the change-management process records nothing, because curves are "market data".
>
> Both readings are defensible and must be reconciled explicitly rather than by silence. The usual resolution distinguishes routine recalibration *within a declared tolerance*, governed by monitoring, from recalibration outside it, which is a change event. The formalism does not choose; it forces you to notice that a choice is being made, and to write the tolerance down.

3 One order

3.1 Four questions that were one question

Four places in a model register ask whether one thing fits where another fitted.

Where	What it asks
An alias move	may this replacement version stand where the incumbent stood?
A fit warrant	does this featureset provide what the kernel declares it reads?
A contract comparison	does this operating contract refine the one it replaces?
A dependency edge	does what the source produces arrive where the target reads it?

In the reference implementation these had four answers: a bespoke variance routine, a hand-written slot loop, a contract refinement check, and — for the fourth — nothing at all. Four implementations of one relation are four opportunities to disagree, and the disagreement is directional: toward permitting more, because a check that wrongly permits produces no bug report. The fourth row is the worst of them, because a dependency graph whose edges were never checked is followed by a blast radius that trusts them.

3.2 The order

Let a *field* be a name, a datatype, a nullability flag and optional numeric bounds; a *schema* a finite set of fields with distinct names.

Definition 3.1 (Field acceptance)**.** Field u *accepts* field v, written $u \succcurlyeq v$, when u and v have the same datatype; u is nullable if v is; u's lower bound is no greater than v's; and u's upper bound is no less than v's, where an absent bound is the widest.

Definition 3.2 (The schema order)**.** For schemas A, B,

$$A \sqsubseteq B \quad \text{iff} \quad \text{for every field } v \in B \text{ there is } u \in A \text{ with the same name and } u \succcurlyeq v.$$

Read $A \sqsubseteq B$ as *A can stand in for B*.

Runs: `core/domain/lattice.py::refines; tests/test_laws.py::TestL20SchemasFormALattice`

A may carry fields B does not; they are simply not read. Each shared field must accept *at least* what B's accepted, because a replacement that rejects an input its predecessor took is a replacement that breaks a caller. This is contravariance in inputs [9, 10], stated as an order rather than as a rule.

> **In plain terms.** "Can stand in for" means: you provide everything the other one provided, and you are no fussier about any of it. Extra things you provide are fine — nobody has to look at them. Being fussier is not fine, because someone was relying on the old tolerance.

Remark 3.3 (The direction is counterintuitive, and reliably so). The order runs opposite to the intuition most readers bring to it. "Refinement" suggests narrowing, and a subtype *is* narrower — in the set of values it denotes. But a schema here is read as a set of *inputs a slot will accept*, and standing in for something requires accepting at least what it accepted. So a wider acceptance is admissible and a narrower one regresses, which is the reverse of the reading the word invites.

This is why the relation is worth writing as an order with a stated direction rather than as a rule of thumb in prose. The two readings of "narrower" are easy to hold at once and impossible to hold consistently, and the failure they produce is silent: an order stated in the intuitive direction admits exactly the replacements that break callers.

Proposition 3.4 (Preorder, and a partial order on canonical form)**.** *$\sqsubseteq$ is reflexive and transitive. It is antisymmetric up to the ordering of fields: if $A \sqsubseteq B$ and $B \sqsubseteq A$ then A and B have the same fields as sets. On schemas presented in name-sorted canonical form — which every constructor in the implementation produces — it is therefore a partial order.*

Runs: `tests/test_laws.py::TestL20SchemasFormALattice::test_the_order_is_a_partial_order`

Proof. Reflexivity is immediate. For transitivity, $\succcurlyeq$ is transitive: datatypes chain by equality; nullability chains by implication; and for the lower bound, if u's bound is absent the condition holds vacuously, and otherwise v's is present and no greater, hence w's is present and no greater — and symmetrically above. For antisymmetry, mutual comparability forces the same names, equal datatypes, equal nullability and, in each direction, an inequality on each bound, so the bounds coincide. □

3.3 It is a lattice

Definition 3.5 (Meet and join)**.** $A \sqcap B$ is the schema on the *union* of the field names, with each shared field widened: nullable if either is, the lesser lower bound and the greater upper bound, an absent bound absorbing. It is defined only when every shared name carries the same datatype in both. $A \sqcup B$ is the schema on the *intersection* of the names that agree on datatype, with each field narrowed: nullable only if both are, the greater lower bound and the lesser upper bound.

Theorem 3.6 (Schemas form a lattice)**.** *On any finite set of field names, $\sqcap$ is the greatest lower bound and $\sqcup$ the least upper bound of $\sqsubseteq$, wherever $\sqcap$ is defined. The empty schema is the top element. There is no bottom.*

Runs: `tests/test_laws.py::TestL20SchemasFormALattice`

Proof. *$A \sqcap B$ is a lower bound.* Every field of A appears in $A \sqcap B$ with bounds no narrower, so $A \sqcap B \sqsubseteq A$, and symmetrically for B.

It is the greatest one. Let $C \sqsubseteq A$ and $C \sqsubseteq B$, and take a field n of $A \sqcap B$. If n occurs in only one of them, C accepts that field by hypothesis. If it occurs in both, C's field at n accepts both, so its datatype agrees, it is nullable whenever either is, and its lower bound is either absent or no greater than each of theirs — hence no greater than their minimum, with the absent case handled by the definition of acceptance, which requires an absent bound on the right to be matched by an absent bound on the left. So $C \sqsubseteq A \sqcap B$.

$A \sqcup B$ is an upper bound. Each field of $A \sqcup B$ is the narrowing of a field present in both, so both accept it.

It is the least one. Let $A \sqsubseteq C$ and $B \sqsubseteq C$, and take $n \in C$. Both A and B carry n with the datatype of C's field and with acceptance at least C's, so n survives into $A \sqcup B$; its narrowed bounds are the tighter of two that each accept C's, hence still accept C's, and it is nullable whenever C's is. So $A \sqcup B \sqsubseteq C$.

Top. The empty schema imposes no condition, so $A \sqsubseteq \varnothing$ for every A.

No bottom. A least element would have to carry every field name that could ever exist; over an unbounded name set no such schema is finite. The structure is a lattice on each finite fragment, which is the only fragment anything inhabits. □

Proposition 3.7 (Order and meet agree)**.** *$A \sqsubseteq B$ if and only if $A \sqcap B = A$ (up to the equivalence of Proposition 3.4), and $A \sqcap (A \sqcup B) = A$. Meet is idempotent, commutative and associative where defined.*

Runs: `tests/test_laws.py::TestL20SchemasFormALattice::test_the_order_and_the_meet_agree, test_absorption_holds, test_meet_is_idempotent_commutative_and_associative`

The link in Proposition 3.7 is what makes the structure a lattice *order* rather than an order with two unrelated binary operations [7]. We do not claim distributivity and do not need it.

> **In plain terms.** Two schemas have a *meet*: the thing that can stand in for both. It has all the fields either of them had, each loosened enough to satisfy both. That is exactly the answer to "what would a single feature set have to provide in order to serve both these models?"
> They also have a *join*: what they agree on. Only the fields both have, each tightened to what both promised. That is the answer to "what may a consumer of either of these rely on?"
> And the empty schema is the top: it demands nothing, so everything can stand in for it.

3.4 The meet is partial, and informatively so

Proposition 3.8 (No meet)**.** *Two schemas whose shared field name carries different datatypes have no lower bound, hence no meet.*

Runs: `core/domain/lattice.py::NoMeet; tests/test_laws.py::TestL20SchemasFormALattice::test_a_meet_that_cannot_ex`

Proof. A lower bound must carry that name with a field accepting both, and acceptance requires datatype equality, which cannot hold for two distinct datatypes. □

The partiality is the useful part. The implementation raises a named error carrying the conflicting slot and both datatypes, rather than returning an empty result — because the caller who asks for a meet has a decision to make, usually *these two models cannot share a featureset*, and an absent value threaded through three layers becomes a silent empty schema somewhere downstream.

Not executed: the meet and join are implemented and their lattice laws are asserted over generated schemas, but no production path calls them yet. The order itself is called from three places; the two operations are the part of Theorem 3.6 that is available rather than deployed, and this paper marks the difference rather than eliding it.

3.5 One relation, four answers

Proposition 3.9 (The four questions are one comparison)**.** *Let* $\mathrm{in}(v)$ *and* $\mathrm{out}(v)$ *be the input and output schemas of a version,* $S(F)$ *the declared schema of a featureset, and* $\mathrm{in}(k)$ *the input schema a kernel declares. Then*

$$\begin{aligned} \textit{version } v' \textit{ may replace } v &\iff \mathrm{in}(v') \sqsubseteq \mathrm{in}(v) \quad \textit{and} \quad \mathrm{out}(v') \textit{ provides } \mathrm{out}(v), \\ \textit{featureset } F \textit{ serves kernel } k &\iff S(F) \sqsubseteq \mathrm{in}(k), \\ \textit{an edge } a \to b \textit{ composes} &\iff \mathrm{out}(a) \sqsubseteq \mathrm{in}(b), \\ \textit{one featureset serves } k_1, k_2 &\iff S(F) \sqsubseteq \mathrm{in}(k_1) \sqcap \mathrm{in}(k_2), \end{aligned}$$

where the second conjunct of the first line is the coarser covariant test on outputs: every name $\mathrm{out}(v)$ *promised is still present at the same datatype.*

Runs: `lines one to three: core/domain/schemas.py::substitutable; core/features/sets.py::FeaturesetRegistry.sati core/registry/composition.py::_check_composes; tests/test_laws.py::TestL20TheSameOrderAnswersEveryQuestion. Line four is law-tested and has no caller.`

The first three lines are answered by one function, and that they are is asserted rather than assumed: a test reads the source of each call site and requires the order to appear in it. That is a crude check and it holds the property that matters, which is not that the three agree today but that they cannot drift apart tomorrow. Three implementations of one relation drift in a predictable direction, and it is not the safe one.

The fourth line is the meet doing work: a featureset that must serve two models must refine their meet, and when the meet does not exist, the honest answer is that no featureset can, with the offending slot named. It is a question the structure can answer and that no production path yet asks — available rather than deployed, and marked as such.

3.6 A refusal that names the remedy

Observation 3.10. The failure of $A \sqsubseteq B$ decomposes into two disjoint sets: names B has that A lacks (*missing*), and names A has that accept less than B's did (*narrowed*). They are different mistakes with different remedies: a missing slot means the wrong featureset was bound; a narrowed one means somebody tightened a constraint without noticing it was a promise.

Runs: `core/domain/lattice.py::Refinement; tests/test_laws.py::TestL20SchemasFormALattice::test_a_missing_field_a`

This is a small thing that changes the character of a control. A refusal that says "incompatible" is a message; a refusal that says "does not provide `turnover`; accepts less than before at `dscr`" is an instruction.

3.7 Edits, and why independence matters

Schemas are not only compared; they are built, by inheriting from parents and applying edits. The edit operations — ADD, DROP, OVERRIDE — generate a monoid acting on schemas under merge-with-rightmost-wins, whose identity is the empty composition.

Proposition 3.11 (Edit laws)**.** *Merge-with-rightmost-wins is associative with the empty map as two-sided identity, and is not commutative. Over this action:* $\textsc{drop}(x) \cdot \textsc{add}(x, \tau) = \mathrm{id}$ *where* x *is absent;* $\textsc{override}(x, \sigma) \cdot \textsc{override}(x, \tau) = \textsc{override}(x, \sigma)$ *with the later winning outright; and two edits naming different slots commute.*

Runs: `tests/test_composition.py; tests/test_laws.py::TestTheEditOperationsCommuteWhenIndependent`

Non-commutativity of the merge is required rather than incidental: a commutative merge has no notion of override, so a child could not correct what it inherits. Commutation of *independent* edits is the property two people editing a shared featureset rely on without knowing it — it is what makes the order in which their edits happened to arrive carry no meaning. It is stated only for independent edits, and deliberately: OVERRIDE then DROP of one slot is not DROP then OVERRIDE, and the second order is refused.

4 Composition, typed — and the risk that does not compose

4.1 An edge is a claim about types

$\mathbf{Para}(\mathcal{C})$ is symmetric monoidal, so a network of models is a string diagram built from generators by sequential composition, parallel composition, copying and discarding. Type-correctness of a wiring is decidable. In a register this becomes a check on the relation that records one model's output being read by another.

Proposition 4.1 (Composition type-check)**.** *An edge* $a \to b$ *asserting that* a*'s output is read as an input by* b *holds only if* $\mathrm{out}(a) \sqsubseteq \mathrm{in}(b)$*. Where it holds, the composite* $b \circ a$ *has the derived signature* $\big(\mathrm{in}(a), \mathrm{out}(b)\big)$*.*

Runs: `core/registry/composition.py::_check_composes, composite_schema; tests/test_laws.py::TestL21FeedsIsCompos`

Three consequences follow from taking the check seriously.

Extra outputs are fine and a missing one is not. By Definition 3.2, an output schema richer than the target reads composes — the surplus is simply unread — while a name the target reads and the source does not produce is a wire to nowhere, and the refusal names it.

The composite's schema is derived, not declared. A composite whose signature somebody wrote down is a composite that can disagree with its parts. This is the same move as Proposition 2.3, one level out.

Not every relation is composition. A register records several ways one model may stand to another, and only some of them describe a wire. "Built from" records lineage and propagates nothing. "Challenger of" and "benchmark for" record how somebody thinks about a model; there is no wire, so there is nothing to type, and treating a challenger as a dependency would inflate every blast radius it appears in. "Calibrated by" propagates — change the calibrator and the answer changes — but does not compose, because solving parameters is not handing an output to an input. Only the input relation is type-checked, and the vocabulary says which is which.

Remark 4.2 (Vocabulary is part of the formalism)*.* The relation was originally called *feeds*. In a bank a feed is market data, a reference file, a nightly drop, so "A feeds B" read as though the governance platform consumed or produced one. It does neither: it moves no data and runs no model, and the edge is a statement about two entries in a register describing a wire that

somebody else's engine carries. The relation is now named for what it asserts. The old spelling is accepted on input and stored under the new name, and is deliberately not published in the vocabulary, because offering two words for one relation invites somebody to believe they mean different things. We record this because a formal account whose terms are read wrongly by its audience has a defect, and the defect is in the account.

Remark 4.3 (A variance choice, recorded as a choice). Proposition 4.1 applies $\sqsubseteq$ — the *input* order — to a pair whose left argument is an output. Presence and datatype are exactly right for composition. The bound and nullability conditions are the input rule applied one level out: they require the producer's declared range to *contain* the consumer's, so a producer with a strictly tighter range is refused. The alternative reading, that producing less is always safe, is what the coarser covariant test in Proposition 3.9 implements for version substitution. The reference implementation therefore holds two variance rules for outputs and applies the stricter one here, on the reading that a consumer fitted over a declared range and then fed a narrower one has had its input distribution changed without a change event. That is a decision and not a derivation, and it is recorded as one.

> **Worked example: the wire that was a drawing**
>
> An ECL stack records that a PD model feeds a provisioning engine. Both entries have existed for two years; the edge is on the diagram in every committee pack.
> The PD model's current version produces `pd_12m`. The provisioning engine's current version reads `pd_lifetime`. Nobody wired the two systems — an adapter in between computes one from the other using a term structure that belongs to a third model, which is not in the register.
> Under an unchecked edge the register reports a two-node dependency and a blast radius that is wrong in both directions: it claims a change to the PD model reaches provisioning directly, and it omits the term structure entirely. Under Proposition 4.1 the edge is refused at the moment somebody records it, with `pd_lifetime` named as the output that does not exist — and the conversation that follows is the one that discovers the third model.

4.2 Aggregate risk

Supervisory guidance asks that risk be assessed "both individually and in aggregate", where aggregate risk "reflects interactions and dependencies among models; reliance on common assumptions, data, or methodologies" [35]. This expectation has precise formal content: it asserts the failure of compositionality.

Definition 4.4 (Risk assignment)**.** Let R be a join-semilattice. A *risk assignment* is a function ρ from morphisms of $\mathbf{Para}(\mathcal{C})$ to R. It is *compositional* if $\rho(g \circ f) = \rho(g) \sqcup \rho(f)$ and $\rho(f \otimes h) = \rho(f) \sqcup \rho(h)$ for all composable pairs.

Compositionality is the assumption made implicitly whenever an aggregate is computed as the maximum or join of constituent ratings, which is standard practice.

Definition 4.5 (Fragility)**.** Fix a network N with generators $G(N)$ and output ports $\mathrm{out}(N)$. For $S \subseteq G(N)$ let $\mathrm{corr}(N, S)$ be the output ports reachable in the diagram from some generator in S. The *fragility* of N is $\mathrm{frag}(N) = \max_{g \in G(N)} |\mathrm{corr}(N, \{g\})|$. A risk assignment is *fragility-sensitive* if $\mathrm{frag}(N) > \mathrm{frag}(N')$ implies $\rho(N) \sqsupset \rho(N')$ whenever the multisets of constituent risks coincide.

Theorem 4.6 (No compositional risk assignment)**.** *Let R be a join-semilattice with at least two elements. No risk assignment is both compositional and fragility-sensitive.*

Proof. Let $a \sqsubset b$ in R. Take generators $A, A' \colon X \to Y$ with $\rho(A) = \rho(A') = a$, and $B, C \colon Y \to Z$ with $\rho(B) = \rho(C) = b$. Let $N_1 = (B \otimes C) \circ \mathrm{copy}_Y \circ A$, in which A's output is copied to both consumers, and $N_2 = (B \circ A) \otimes (C \circ A')$; both have two output ports. The multisets of

constituent risks coincide, both being $\{\{a, b, b\}\}$. But $\mathrm{corr}(N_1, \{A\}) = \mathrm{out}(N_1)$ so $\mathrm{frag}(N_1) = 2$, while every generator of N_2 reaches exactly one port so $\mathrm{frag}(N_2) = 1$. If ρ is compositional then $\rho(N_1) = a \sqcup b = \rho(N_2)$, contradicting fragility-sensitivity. □

Runs: the reference implementation carries the lax structure of Corollary 4.8 without ever producing an aggregate ρ as a *number*, which is what Theorem 4.6 says it cannot honestly do. The join over the tier lattice is computed; the interaction term is a set of *named obstructions* — an unassessed component sits at the top of the lattice, so silence escalates rather than passing — and `shared_dependencies` supplies them. The theorem is respected by refusing to state a magnitude, not by declining to compose. This is law L-14, and it runs in `core/risk/aggregate.py`.

> **In plain terms.** Two banks each run one curve model and two pricers. Bank A uses a single curve to feed both. Bank B built two curves independently, one for each.
> Rate the models individually and you get identical answers: same curve rating, same two pricer ratings. So any method computing the total from the individual ratings gives both banks the same answer.
> But Bank A has a single point of failure and Bank B does not. That is the entire theorem: *the risk of the whole is not a function of the risks of the parts, because part of it lives in the wiring.*

Remark 4.7 (The obstruction is the copy map). N_1 and N_2 differ in exactly one respect: N_1 uses copy_Y. Shared dependency *is* the comonoid copy map. In a Cartesian category copying is implicit and structurally invisible, so the distinction cannot be expressed at the level of the algebra — a concrete reason to prefer a Markov category as the ambient setting even when the artefacts under management are deterministic.

Corollary 4.8 (Laxity is necessary)**.** *Any fragility-sensitive risk assignment is at best lax monoidal, $\rho(g \circ f) \sqsupseteq \rho(g) \sqcup \rho(f)$, with the discrepancy attributable to shared constituents. We call $\rho(N) \ominus \bigsqcup_{g \in G(N)} \rho(g)$ the* interaction premium.

The practical reading is that a governance system should report the join of constituent risks *and* the composite, and attribute the difference: to common parameter objects (Proposition 2.11), common inputs, common fitting evidence, common methodology. The first three are computable from a typed graph; the fourth is not, and we do not pretend otherwise.

5 One operator

5.1 Two clocks

A fact carries two independent time coordinates: the *event time* at which it held in the world, and the *ingest time* at which the recording system learned it [16, 17]. Conflating them produces look-ahead leakage, which is the dominant silent failure mode in the domain [18]: an artefact fitted on facts that had not been recorded when the decision was taken assesses well and performs badly, and the discrepancy is attributed to degradation rather than to leakage.

The condition itself is folklore. What is not usually stated is that the *read* is an operator with laws, and that the laws are where reproducibility lives.

5.2 The read as an operator

Definition 5.1 (Point-in-time read)**.** Let R be a finite set of records, each carrying event and ingest times in a totally ordered set. For a label time ℓ and an observation time a put

$$\mathrm{Adm}(R, \ell, a) \;=\; \{\, r \in R \;:\; r.\mathrm{event} \le \ell \;\wedge\; r.\mathrm{ingest} \le \min(\ell, a) \,\},$$

$$\mathrm{AsOf}(R, \ell, a) \;=\; \operatorname*{arg\,max}_{r \in \mathrm{Adm}(R,\ell,a)} \big(r.\mathrm{event},\, r.\mathrm{ingest}\big)$$

under the lexicographic order, undefined when $\mathrm{Adm}(R, \ell, a) = \varnothing$.

Runs: `core/features/assembly.py::TrainingSetBuilder.latest_admissible`

Both bounds are present because they refuse different things. The event bound is what the world had done by the moment of the decision. The ingest bound is what could have been *known* at that moment — and it is bounded by $\min(\ell, a)$ rather than by a alone, because a is a single scalar for a whole assembly and is usually much later than any individual label.

Proposition 5.2 (Four properties)**.**

1. Idempotent. *If* $r = \mathrm{AsOf}(R, \ell, a)$ *is defined then* $\mathrm{AsOf}(\{r\}, \ell, a) = r$.
2. Commutes with projection. *For any projection* π *that deletes non-clock columns,* $\mathrm{AsOf}(\pi R, \ell, a) = \pi\,\mathrm{AsOf}(R, \ell, a)$.
3. Monotone in a. *If* $a \le a'$ *then* $\mathrm{Adm}(R, \ell, a) \subseteq \mathrm{Adm}(R, \ell, a')$.
4. Saturating at ℓ. *For all* $a, a' \ge \ell$, $\mathrm{Adm}(R, \ell, a) = \mathrm{Adm}(R, \ell, a')$ *and hence* $\mathrm{AsOf}(R, \ell, a) = \mathrm{AsOf}(R, \ell, \ell)$.

Runs: `tests/test_laws.py::TestL10TheAsOfOperator`

Proof. (1) r is admissible, so it lies in the admissible set of the singleton, of which it is the maximum. (2) Admissibility and the ordering key are functions of the two clocks alone, which π preserves; ties are broken by position, which π also preserves. (3) $\min(\ell, a) \le \min(\ell, a')$, and the event bound does not involve a. (4) $\min(\ell, a) = \ell = \min(\ell, a')$, so the two admissible sets are the same set, and the arg max over it is the same record. □

Remark 5.3. Property (3) is monotonicity of the admissible *set*, not of the chosen record: a later observation time may cause a different row to be selected. It says only that nothing which was knowable stops being knowable. Adm is likewise monotone in ℓ, by the same argument; that extension is not asserted in the test suite and is marked accordingly.

Not executed: monotonicity in ℓ; the suite asserts monotonicity in a only.

Corollary 5.4 (Saturation is the reproducibility guarantee)**.** *A row assembled at any observation time at or after its label is the row that would be assembled at the label, and at every later observation time, whatever arrives in between. Reproducibility of a training set is therefore a property of the operator rather than a property of the discipline of whoever re-runs it.*

Runs: `tests/test_laws.py::TestL10TheAsOfOperator::test_it_saturates_at_the_label_and_that_is_the_reproducibility`

> **In plain terms.** Every fact has two dates: when it was true, and when you found out. Ignore the second and you build training sets containing information that did not exist when the decision was made. The model looks excellent in testing — it has been shown the answers — and fails in production, and the post-mortem blames drift.
> The rule is: use the latest fact that was both true by the decision date *and known by the decision date*. The second half is the one people drop, usually by bounding it with "now" instead.
> And the reason it is worth stating as an operator rather than as a rule is the last property. Because the knowledge bound is the *earlier* of the decision date and the assembly date, re-running the assembly later cannot change the answer. Without that, a re-run quietly improves on the original — which is the least useful kind of reproducibility, because the numbers then agree with nothing, including themselves.

Worked example: the filed accounts that were restated

A borrower files Q1 accounts dated 31 March. They land in the warehouse on 20 May. The bank declines an application on 15 June. In August the borrower restates those accounts downward, from a debt-service ratio of 1.20 to 0.40.

Both rows are true and both belong in the store: same event time, different ingest times.

Read with one clock, the training row for the June decision gets 0.40, because that is the current value for Q1. The model learns to predict defaults using a number that exists *because* the default already happened.

Read with two, it gets 1.20 — and by Corollary 5.4 it still gets 1.20 when the set is rebuilt in 2028, after two more restatements. And the row labelled in September correctly gets 0.40, because by then the restatement was knowable.

5.3 A corollary about where the data may live

Corollary 5.4 is usually read as a statement about how to *query* a store. It is also a statement about which stores may be queried at all, and the second reading is the operational one.

The corollary holds because the knowledge bound is $\min(\ell, a)$ over rows that carry both clocks and are never amended in place: a restatement is a new row with a later ingest time, so the earlier read remains derivable. A source that overwrites — an ordinary warehouse table, a view maintained by a nightly job, a file replaced on a schedule — carries neither property. Its 20 May row is simply gone once the August restatement lands, and no operator applied at read time can recover what the June decision saw. The read does not fail; it returns 0.40 and says nothing, which is the failure mode Section 5 exists to eliminate.

So a platform that binds a featureset slot directly to an external table has not implemented Proposition 5.2; it has implemented a query that usually agrees with it. The two are indistinguishable until the first restatement, and then indistinguishable again afterwards, because nothing records that they diverged.

> The consequence for a build is one line: *copy, do not connect.* An external source is pulled — read once, bitemporalised, and written into storage the platform controls as an immutable, versioned snapshot — and models read the snapshot. Connecting is cheaper and gives up the corollary.
>
> This is not a preference for one storage technology. It is the observation that Proposition 5.2 constrains its argument as much as its definition: an operator defined over rows that can be overwritten is defined over the wrong object, and the guarantee it appears to provide is a guarantee about a table that no longer exists.

Runs: `core/features/sources.py::SourceRegistry.pull; tests/test_feature_sources.py::TestAPullProducesAnOrdinaryV`

5.4 Two properties the operator has to carry beyond itself

Proposition 5.2 is a statement about a function. A governance platform does not execute the training assembly — an engine somewhere else does — and it checks the assembly with a second computation of its own. So the operator has to hold in two more places than the one it is defined in, and each of those is a property worth stating on its own.

The published rule is the applied rule. The point-in-time predicate is published as a string, in every feature-set plan and every execution warrant, because the engines that implement it are not the ones the platform runs. That string is a *contract*, and the property required of it is agreement: for every set of records, the rows the published predicate admits are the rows the operator admits.

The way to check that is to *execute* the published string against the same records and compare with Definition 5.1, which is what the law test does. Comparing the text instead —

asserting that the published rule contains some expected substring — is a weaker check in the specific way that matters: it passes on a rule that reads plausibly and admits different rows. An engine faithfully implementing a published contract that disagrees with the platform's own read produces an unreproducible training set with two correct-looking implementations and no way to tell which one is wrong.

The independent recomputation bounds the clock identically. Assembly is verified by a second computation that deliberately does not reuse the assembly path, so that agreement between them carries information. The value of that arrangement rests entirely on the two routes implementing the *same* operator: a verifier that bounds the ingest clock by the observation time while the assembly bounds it by $\min(\ell, a)$ disagrees on every set assembled after its labels matured, which is the ordinary case, and therefore reports a mismatch on a correct assembly.

That failure mode deserves naming, because it is worse than the one it looks like. A control that reports a violation where there is none is not a conservative control; it is one that teaches whoever reads the report to discount it, and it does so precisely for the population it was built to examine. Stating the operator once and requiring both routes to be it is what removes the possibility.

A change of storage is not licence for a second implementation. Definition 5.1 is defined over records and says nothing about the medium holding them. A platform that later gains a second storage backend nevertheless acquires, at that moment, the opportunity to write the predicate a second time — and the local pressure is toward doing so, because the second backend arrives with its own idioms and the existing function is written against the first. The reference implementation now supports two table formats, and the point-in-time read is the *same function object* under both. That is asserted as identity rather than as agreement of results, which is the stronger claim in the way that matters here: two implementations agreeing on the cases somebody thought to test is compatible with their disagreeing on the case that arises in production, and the disagreement would be invisible because both sides would be reporting a correct-looking answer. The general form is the moral of Section 3: the number of implementations of a relation is a design variable, and every value above one is a standing decision to permit disagreement.

Runs: `db/delta_store.py::point_in_time; db/iceberg_store.py; tests/test_iceberg_store.py::TestBothStoresBehaveTh`

5.5 The input object, made an object

The operator reads records; something has to say which records. Treating the input object X with the same seriousness as P gives it a name and a version.

A *featureset* declares a schema of named, typed slots — and that schema, by Proposition 3.9, is what a kernel is defined over. A *version* of the featureset *fills* the schema, binding each slot to a specific feature and to the pinned version of the view supplying its values. The separation does three things. Swapping which feature fills a slot does not change the model's input space; it changes what the model was fitted on, which is a different event with a different control. A version that cannot fill the schema is refused, because adding a slot is a change to X and therefore a model change rather than a data change. And because every binding pins a version, one featureset version resolves to the same bytes on every read — a stable identifier over moving contents being the failure this design exists to prevent.

6 One polynomial

6.1 Evidence is a derivation, not a log

Assurance evidence forms a directed acyclic derivation: fitting evidence supports a fitting run, which with code and environment supports a version, which supports test outcomes, which support a validation conclusion, which supports an authorisation, which supports a deployment. Governance claims are derived from base evidence by conjunction (joint dependence) and disjunction (alternative routes).

Annotate each base item with an element of a commutative semiring $K = (K, \oplus, \otimes, 0, 1)$ and propagate with $\otimes$ for joint dependence and $\oplus$ for alternatives. This is exactly the setting of provenance semirings [11].

Proposition 6.1 (Universality)**.** *Let $\mathbb{N}[X]$ be the semiring of polynomials with natural coefficients over evidence identifiers X. For every commutative semiring K and valuation $\nu\colon X \to K$ there is a unique semiring homomorphism $h_\nu\colon \mathbb{N}[X] \to K$ extending ν, and for every derivation d,*

$$\mathrm{eval}_K(d, \nu) \;=\; h_\nu\big(\mathrm{eval}_{\mathbb{N}[X]}(d)\big).$$

Runs: `core/evidence/semirings.py::POLYNOMIAL, pushforward; tests/test_laws.py::TestL9TheProvenancePolynomialIsU`
`-- checked over 200 randomly generated derivation DAGs against five semirings`

Proof. $\mathbb{N}[X]$ is the free commutative semiring on X [11]; the displayed equation is the universal property applied to the evaluation, which uses only $\oplus$ and $\otimes$. □

The content of Proposition 6.1 for a governance system is that the same traversal answers a different question for each semiring, and that this is a theorem rather than a coincidence: where a direct evaluation and a pushforward disagree, one of the two routes is not a homomorphism, which is a fact about the structure and not about the traversal.

Semiring	$\oplus$, $\otimes$	**Governance question answered**
$\mathbb{B}$	$\vee$, $\wedge$	Is the claim supported at all?
$\mathbb{N}$	$+$, $\times$	How many independent derivations corroborate it?
PosBool(X)	absorptive $\cup$, pairwise $\cup$	Which minimal evidence sets suffice?
$\mathbb{N}[X]$	$+$, $\times$	Exactly how was it derived?
$([0,1], \max, \times)$	max, $\times$	With what confidence?
$(\mathbb{R}^+ \cup \{\infty\}, \min, +)$	min, $+$	At what least cost can a gap be closed?
$(2^{\mathrm{Regimes}}, \cap, \cup)$	$\cap$, $\cup$	For which regimes is the evidence admissible?

Remark 6.2 (Which semiring the minimal-support one is). The implemented "why" semiring applies absorption — a set of supports is reduced to its minimal members, so $a \oplus ab = a$. With absorption this is the semiring of positive Boolean expressions PosBool(X) rather than the Why(X) of [13], which is idempotent but not absorptive; the two sit at different levels of the provenance hierarchy [11, 14]. The distinction matters for what is retained: PosBool answers "what is the smallest set I could show an examiner", and discards the fact that a claim had a second, larger, independent route. $\mathbb{N}[X]$ retains both — coefficients count distinct derivations and exponents count repeated use of one fact — which is precisely the information Boolean provenance throws away, and the reason to evaluate in the free object once rather than in each semiring separately.

Runs: `tests/test_laws.py::TestL9TheProvenancePolynomialIsUniversal::test_the_polynomial_keeps_what_boolean_prove`

In plain terms. A semiring is just "a way of doing arithmetic": something playing the role of plus, something playing the role of times.
Tracking evidence has a fixed shape — this rests on that test AND that approval; this requirement can be met by this route OR that one. Keep the shape and swap the arithmetic and you get a different answer to a different question from the same computation. True/false arithmetic tells you whether the claim holds; probability tells you how much to trust it; least-cost tells you the cheapest way to close a gap; sets-of-evidence tells you what to put in front of an examiner.
The polynomial is the version that keeps everything, so all the others can be read off it afterwards rather than recomputed.

6.2 One layer across: derived features

A derived feature is a term: $Z = f(X, Y)$, computed from other features by a small expression language. The same construction applies, and gives a data-engineering rule the status of a theorem.

Proposition 6.3 (The ingest clock is a homomorphism)**.** *Let* $\mathrm{prov}(Z) \in \mathbb{N}[X]$ *be the evaluation of Z's derivation with each base feature sent to its own variable, and let* ν *send each base feature to its ingest time. Then the time at which Z became knowable is* $h_\nu(\mathrm{prov}(Z))$ *computed with* $\oplus = \otimes = \max$*; equivalently, the maximum ingest time over the free variables of* $\mathrm{prov}(Z)$*.*

Runs: `core/features/derived.py::provenance, rests_on, knowable_at; tests/test_laws.py::TestL9ReachesDerivedFeatu`

The upgrade is worth stating plainly. "The ingest time of a derived feature is the maximum over its inputs" was documented as arithmetic, on the grounds that arithmetic cannot be forgotten. It is stronger than that: it is the image of the provenance polynomial under a valuation, and a homomorphism has no exceptions to forget. Every route to the number goes through the same object.

Two further questions fall out of the same polynomial:

What does this rest on? The free variables — and these are deliberately *not* the transitive ancestor set, because the two answer different questions. The ancestor walk returns every ancestor, derived ones included, which is the right answer to *what breaks if this changes.* The free variables are the base features and nothing else, which is the right answer to *what data does this ultimately read.* The second is the first minus its derived members, and the identity is asserted rather than assumed: two routes to one answer are worth having only while they agree.

Does it touch the label? Membership of the label's variable in the free variables. A feature derived from a derivation of the label is still the label, and leakage detection becomes a membership test rather than a graph walk with a depth limit.

Runs: `tests/test_laws.py::TestL9ReachesDerivedFeatures`

6.3 A negative result: currency is not a semiring valuation

The list of semirings above is shorter than the design it came from. That design named a currency valuation, $(\max, \max)$, answering "as of when is this claim current?". It is not a semiring, and no choice of constants repairs it.

Proposition 6.4 ($(\max, \max)$ is not a semiring)**.** *Let* $(K, \leq)$ *be totally ordered and let* $\oplus = \otimes = \max$*. If* $(K, \oplus, \otimes, 0, 1)$ *is a semiring then* $|K| = 1$*.*

Runs: `core/evidence/semirings.py::FRESHNESS; tests/test_laws.py::TestL9TheProvenancePolynomialIsUniversal::test_`

Proof. An identity for max must be a least element, so 0 is the bottom of K; likewise 1 is the bottom of K; hence $0 = 1$. In any semiring $0 = 1$ gives $a = a \otimes 1 = a \otimes 0 = 0$ for every a. Independently, the annihilation law fails directly: $0 \otimes a = \max(\bot, a) = a \neq 0$ for $a \neq 0$. □

The practical consequence is specific and worth knowing rather than hiding. Because the zero does not annihilate, a claim resting on a *missing* fact reports the currency of the facts that are present, rather than reporting that it has none. A structure that answers "as of when" for a claim it cannot in fact support is exactly the kind of instrument that reads as reassurance.

The repair is not a different pair of operations. Currency is an *aggregate* over provenance rather than a valuation of it, and aggregates over semiring-annotated data require semimodule structure rather than a semiring on the annotations [15]. We have not built it, and record the gap in Section 8 rather than describing an intention as a result.

> **In plain terms.** One of the six "arithmetics" turned out not to be one. If you use "the later of the two" for both plus and times, then the thing that is supposed to represent *no evidence at all* stops behaving like zero: combine it with a real timestamp and you get the timestamp back, not nothing.
> So a claim resting on a missing document reports a date, and the date looks fine. We found this by writing the test that asserts the universal property, which the currency arithmetic then failed. It is now a test that asserts the failure, so that nobody quietly re-includes it.

6.4 Citation soundness

Proposition 6.5 (Citation soundness)**.** *Let c be a claim with derivation d_c over evidence identifiers X and let $S \subseteq X$ be a cited set. Define $\nu_S(x) = \top$ iff $x \in S$. Then S supports c if and only if the evaluation of d_c in $\mathbb{B}$ under ν_S is $\top$; equivalently, iff S contains all the variables of some monomial of* $\mathrm{eval}_{\mathbb{N}[X]}(d_c)$.

Proof. By Proposition 6.1, evaluation in $\mathbb{B}$ under ν_S is h_{ν_S} applied to the $\mathbb{N}[X]$ evaluation. A polynomial evaluates to $\top$ under ν_S exactly when some monomial has all its variables in S. □

Not executed: the pushforward and the Boolean semiring both execute and are covered by `TestL9TheProvenancePolynomialIsUniver` the application to generated document text is design.

This is a small observation with a large practical consequence, taken up in Section 9: over an annotated derivation structure, "does this citation support this claim" is an evaluation, not a further model call.

7 What must still be declared

A paper arguing that four facts derive owes an account of the ones that do not. Three kinds are worth distinguishing, and only the third is irreducible.

7.1 Derived from a declared rule

Observation 7.1 (Constitutive rules)**.** Let $q = f(\Phi)$ be a derived governance quantity: a risk classification, a control requirement, a scope determination. Computing q is deterministic and involves no judgement. The judgement lives in two other places: supplying Φ, and choosing f. And f *constitutes* the standard, so there is no independent specification against which a proposed f could be checked.

Risk tiering is the clearest instance. Let M be a materiality lattice, C a complexity lattice and T a finite chain of tiers.

Proposition 7.2 (Monotone classification)**.** *If $\tau\colon M \times C \to T$ is monotone then increasing exposure, criticality of purpose or complexity never lowers the assigned tier.*

Runs: `tests/test_risk.py::TestTauMonotonicity`

Proposition 7.3 (Control adequacy)**.** *Let* Ctrl *be a lattice of control sets ordered by strictness and let* $\mathrm{req}\colon T \to \mathrm{Ctrl}$ *and* $\sup\colon \mathrm{Ctrl} \to T$ *form a Galois connection,* $\mathrm{req}(t) \sqsubseteq c \iff t \sqsubseteq \sup(c)$*. Then "the applied controls meet the tier's requirements" and "the tier is within what those controls can defend" are the same statement.*

Runs: `tests/test_risk.py::TestControlAdjunction`

Monotonicity is trivial as mathematics and consequential as an assurance property: it is exactly the guarantee an examiner seeks — *nothing we can learn that makes a model more consequential will move it into a lighter regime* — and exactly what an unconstrained scoring formula fails to provide. The adjunction means a control deficiency and an inflated classification are one defect seen from two sides. But τ, req and the lattices themselves are declared, and rightly so.

7.2 Declared, but with a consistency oracle

The same artefact may be outside one regime's governed population, inside a second's, high-risk under a third and a financial-reporting key control under a fourth. These are not four values of one attribute but four logical systems, each with its own vocabulary and its own notion of what makes a sentence true of an inventory state. Modelling each as an *institution* [20, 21] — signatures, sentences, models, satisfaction — makes admitting a regime the exhibition of a module rather than a migration.

Proposition 7.4 (Determination stability)**.** *For an institution comorphism* $\varrho = (\Phi, \alpha, \beta)$ *from a regime institution to the core inventory institution, and for every core state* M' *and regime sentence* ϕ*,*

$$M' \models' \alpha_\Sigma(\phi) \quad \iff \quad \beta_\Sigma(M') \models_\Sigma \phi.$$

A determination computed on the core inventory using the translated obligation agrees with the one computed natively in the regime's vocabulary.

Runs: `core/regimes/translation.py::satisfaction_condition -- enforcing: a regime whose encoding fails it cannot be activated. The quantifier is over six probe states spanning the corners rather than over generated ones, and that weakening is stated rather than hidden.`

Corollary 7.5 (Falsifiable encoding test)**.** *Any observed disagreement between the two sides witnesses a defect in the encoding rather than a genuine normative conflict.*

The encoding is a declaration — a claim about a regulation, not the regulation — but it is a declaration with an oracle attached, which is a strictly better position than a checkbox with a comment. The same shape recurs for deontic consistency: whether a regime obliges and forbids the same term is decidable over the sentences whose form is declared, and the sentences whose form the checker cannot read are *named* rather than assumed consistent, because a check that silently ignores what it cannot judge reports success for exactly the cases it was least able to judge.

Runs: `core/regimes/sentences.py::deontic_conflicts, undecidable; tests/test_laws.py::TestL16NoRegimeObligesAndFo`

7.3 Irreducibly declared

Concluding a validation, granting an approval, accepting residual risk, and choosing τ: these are not weakly-checkable tasks conservatively withheld from derivation. They have no notion of correctness independent of the authority exercising them. Section 9 takes up what follows.

7.4 Two constructions that live on the boundary

Contracts. A *contract* (A, G) pairs an assumption on the environment with a guarantee conditional on it [22]. The mapping into the domain is exact and clarifies a term supervisory texts use without defining: a model's *operating boundary* is A and its performance envelope is G. Use outside the boundary is a contract violation, checkable at execution, and when A fails G is void — a considerably stronger statement than "the input looked unusual". Substitution becomes refinement, which by Proposition 3.9 is the platform's one order applied to contracts. A and G are declared; whether one refines another is derived.

Probe-relative identity, and artefacts that remember. Behavioural equivalence is only as strong as the probe set that witnesses it. Writing Π for a finite probe set and $\equiv_\Pi$ for agreement on it, the three standard version increments have formal content: PATCH asserts the same signature and $\equiv_\Pi$; MINOR the same signature with a different inhabitant of P (Proposition 2.10); MAJOR a different signature. The honest clause is that $\equiv_\Pi$ refines monotonically in Π, so a claim of equivalence is exactly as strong as Π is rich, and a system should record Π with every such claim.

For artefacts that carry state this is not a caveat but a defeater. A stateful kernel $f\colon P \otimes S \otimes X \to S \otimes Y$ has, for fixed parameters, a behaviour family $\mathsf{beh}_n\colon X^{\otimes n} \to Y^{\otimes n}$.

Proposition 7.6 (Single-shot probes are unsound for stateful artefacts)**.** *There exist stateful kernels v_1, v_2 of the same signature with $\mathsf{beh}_1(v_1) = \mathsf{beh}_1(v_2)$ and $\mathsf{beh}_2(v_1) \neq \mathsf{beh}_2(v_2)$. Moreover, for every n there exist kernels agreeing on all probes of input length at most n and differing at $n+1$.*

Proof. For the first, let $X = \{*\}$, $Y = \{0,1\}$; let v_1 emit a fresh fair coin on every call, and v_2 draw one fair coin at start-up and repeat it. Then beh_1 agree and are uniform, while $\mathsf{beh}_2(v_1)$ is uniform on $Y^{\otimes 2}$ and $\mathsf{beh}_2(v_2)$ puts mass $\frac{1}{2}$ on each of $(0,0)$ and $(1,1)$. For the second, take $S = \{0, \dots, n\}$ with each call incrementing the state; let both emit 0 while the state is below n, and thereafter let v_1 emit 0 and v_2 emit 1. □

Not executed: Π is recorded per version; probe *shape* is not constrained, so nothing currently refuses a length-one probe set for a stateful artefact.

> **Worked example: a seed policy that was a patch release**
>
> A counterparty simulation engine draws random numbers from a generator seeded once at process start. A performance change seeds it per request instead, removing a lock and halving latency. Nothing about the model, the calibration or the parameters changes; the release is labelled a patch.
>
> The nightly suite runs eleven thousand single-trade valuations and reproduces every one within tolerance, because a single valuation averages over paths either way. What changes is the correlation *between* valuations in a batch: two trades in one netting set were previously simulated against common paths and now are not. Netting-set exposure moves materially, in the direction of understatement.
>
> Eleven thousand probes could not detect it, and no number of probes of that shape could have. The defect is not that the suite was too small; it is that every element of it had length one.

8 Laws as the criterion of seriousness

8.1 The discipline

A foundation that is documented but unenforced decays into ornament within a small number of releases, and thereafter misleads. The discipline we propose is that each result be restated as

an executable law and checked by property-based testing against generated inputs [26], with failure treated as a build failure — and, crucially, that each law be tested *as stated* rather than as the implementation happens to behave. A test written from the code proves only that the code agrees with itself.

8.2 What runs, and what does not

The reference implementation states twenty-one laws. Eighteen execute; three do not.

Law	Statement	State
L-1	lifecycle is a free category over declared transitions	runs; found a second initial state
L-2	a version digest never moves	runs, over the manifest rather than the row
L-3	a determinism claim is checked or refused	runs
L-4	tiering is monotone (Proposition 7.2)	runs
L-5	control adequacy is an adjunction (Proposition 7.3)	runs, exhaustively
L-6	summary soundness: $c \sqsubseteq \gamma(a)$	*not built*
L-7	contract refinement on substitution	runs, and enforces at alias move
L-8	satisfaction condition (Proposition 7.4)	enforces, over probe states
L-9	provenance homomorphism (Proposition 6.1)	runs, over generated DAGs
L-10	the point-in-time read and its four properties (Proposition 5.2)	runs, and enforces statically
L-11	lens laws for documents	*not built*
L-12	schema variance (Proposition 3.9)	runs, and enforces
L-13	evidence gluing within a consistency radius	*not built*
L-14	lax monoidality of risk (Corollary 4.8)	runs, and enforces
L-15	fibration totality (Proposition 2.6)	runs, and gates start-up
L-16	no obligation contradiction	runs, and enforces
L-17	contract–serving agreement	runs, over declared servings
L-18	no personal data inline in evidence nodes	runs, and enforces
L-19	composition is a monoid (Proposition 3.11)	runs
L-20	schemas form a lattice (Theorem 3.6)	runs, and enforces
L-21	composition type-checks (Proposition 4.1)	runs, and enforces

The three that do not run are named with their reasons, in the test file as well as in the table, because a gap recorded only in a document is a gap somebody has to go looking for. L-6 needs a replay that checks a document's quantitative claims against the register; the replay exists for validation episodes and not for documents. L-11 needs a *put*: the compiler regenerates whole documents, so there is no round trip, and building one merely to satisfy a law would be building the wrong thing. L-13 has no implementation; no consistency radius is computed anywhere.

Two laws left this list, and how they left it is the more interesting half. L-14 was said to need composite warrants and an aggregate risk *function*; what it actually needed was to stop trying to be a number. The lax interaction term is now carried as a set of *named obstructions* — what has not been assessed about the composite — joined over the tier lattice, with an unassessed component sitting at the top. A composite risk figure that looks like a measurement and is not one is worse than the absence of one, so the magnitude is still deliberately unbuilt while the law itself is enforced.

L-17 was said to need an online serving store. It did not: it was blocked on the wrong thing. The design states that the platform will not sit on the serving path, so a store would have contradicted it. The engine instead *attests* which feature namespaces it read, and the platform compares that against what the version's contract pins — training–serving skew becomes detectable without the platform being on the request path, and *unattested* is its own state

rather than a silent pass. What a store would still buy is observation rather than attestation, which is a smaller claim than the one this paragraph used to make.

A property test enforces that this list and the public table agree: if the table claims a law runs, something must run it.

Runs: `tests/test_laws.py::TestTheLawsStillNotExecutable`

8.3 What an executable law delivers that a documented one does not

The difference is not diligence. A statement in a document and the code it describes are two accounts of one rule, and two accounts of one rule diverge — not because anybody decides to let them, but because only one of the two is executed, and the executed one is the only one that anything pushes back on. An executable law is the comparison between them, performed by a machine, on a schedule nobody sets.

Four properties follow, and each is a property of the arrangement rather than of the people in it.

1. *A rule stated once cannot disagree with itself.* Proposition 3.9 is the instance: four questions, one function, and a test that reads each call site to keep it that way.
2. *A published contract can be executed rather than quoted.* Section 5.4 is the instance. Where a rule is handed to a system this one does not run, the executable form of the law runs the published rule and compares answers, which is a check on the contract rather than on its spelling.
3. *Two routes to one answer can be required to be the same route.* Independent recomputation is worth its cost only under that requirement, and the requirement is a law rather than a convention.
4. *A property that holds on the examples somebody chose is not the property.* Laws are checked against generated inputs [26], and stated as the law is stated rather than as the implementation behaves — a test written from the code proves only that the code agrees with itself.

The fourth is the one that decides whether the other three are real. It is also the reason the five unexecuted laws above are listed rather than described: the property they lack is not documentation.

9 Automation is the same boundary

9.1 Oracles

The constructions above were introduced to secure governance properties. They have a second use, which was not the design intent and which we now regard as the most practically consequential: several of them are *decision procedures*, and a decision procedure is exactly what makes machine-generated output safe to accept.

Definition 9.1 (Oracle)**.** An *oracle* for a task T with outputs in O is a decidable predicate $\mathsf{ok}_T \colon O \to \mathbb{B}$, computable from the formal structure and from data independent of the output, such that $\mathsf{ok}_T(o)$ holds only if o is correct for T.

Proposition 9.2 (Automation admissibility)**.** *If T is oracle-backed then the soundness of "generate with any procedure g, accept iff ok_T" is independent of g: no incorrect output is accepted. The generator's error rate determines throughput, not correctness. If T is not oracle-backed, the correctness of the output is exactly the correctness of the generator.*

Proof. Immediate. In the first case acceptance implies ok_T, which implies correctness by Definition 9.1, and the rejected fraction is a throughput quantity. In the second there is nothing between g and acceptance. □

The observation this paper adds is that the oracle boundary is not a new boundary. It is the derive/declare boundary of Sections 3 and 5 to 7, seen from the other side: a fact that derives from structure has an oracle, namely the derivation; a fact that constitutes the standard has none, by Observation 7.1, because there is nothing independent for it to be checked against.

Task	**Oracle**	**From**
Encode a regulatory regime	the satisfaction condition	Proposition 7.4
Assert two versions behave alike	$\equiv_\Pi$ on the declared probe set	Section 7
Substitute one version for another	$\sqsubseteq$ on inputs, provision on outputs	Proposition 3.9
Wire one model into another	$\mathrm{out}(a) \sqsubseteq \mathrm{in}(b)$	Proposition 4.1
Bind a featureset to a kernel	$S(F) \sqsubseteq \mathrm{in}(k)$	Proposition 3.9
Assemble fitting evidence without leakage	the point-in-time operator, recomputed independently	Proposition 5.2
Cite evidence for a written claim	Boolean pushforward of the derivation	Proposition 6.5
Propose a remediation plan	*none needed* — computed in the tropical semiring	Section 6
Propose a probe or a query	it runs and discriminates, or it does not	—
Choose the tiering rule τ	*none exists*	Observation 7.1
Conclude a validation	*none exists*	Observation 7.1
Accept residual risk	*none exists*	Observation 7.1

> **In plain terms.** If you can *check* an answer, it does not matter much what produced it: a wrong answer is caught and discarded, and the only cost is wasted effort. If you cannot check it, then trusting the answer is trusting the producer, and no amount of process changes that.
> So the useful question about automating a governance task is not "is the model good enough?" but "do I have a check?" — and this paper's answer is that you have a check exactly where the fact was derived rather than declared. The two questions were the same question all along.

The first row inverts an apparent weakness. Encoding a forty-page supervisory statement into signatures and sentences is expensive expert work, and that cost is the practical objection to the institutional approach. It is also a task language models are unusually well suited to, and the output is checkable: a proposed encoding either satisfies Proposition 7.4 or it does not. Generation becomes cheap, verification mechanical, and the expert's role changes from authoring to adjudicating an encoding that has already passed a consistency test.

9.2 The reviewer is part of the system

Proposition 9.2 guarantees that no incorrect output is accepted *by the oracle*. In a governance process acceptance is an act performed by a person, and the composite that actually runs includes them. The uncovered part has a documented failure mode: machine-drafted text is fluent, fluency is read as care, and reviewers of polished artefacts find fewer defects than reviewers of rough ones. We cannot dissolve this, and the framework does three things short of dissolving it.

It determines the division of attention formally. Where an oracle exists, the reviewer is not checking correctness — Proposition 9.2 disposed of that — but authority, which by Observation 7.1 admits no oracle. Marking that boundary in the interface is a mitigation available only because the boundary is formally determined rather than a matter of taste.

It supplies constructed rather than rhetorical objections. An uncited claim is a failing conjunct under Proposition 6.5. A refused wire names the field that does not compose. A featureset

that does not serve a kernel names the slot that is missing or narrowed (Observation 3.10). These survive being stated in prose as fluent as the draft's, because none of them is a matter of emphasis.

It makes one thing measurable that otherwise is not. A claim in a generated document is supported by a set of nodes in the derivation structure. Whether those nodes were retrieved during the review is a fact about the session rather than an inference about the reviewer, and a pattern of attestations to claims whose support was never opened is the clearest available indicator of the failure mode in question. The sharpest signal is the rate at which a reviewer accepts drafts the oracle refused: exactly the population where person and procedure disagree, and a quantity that exists only in a system with a procedure to disagree with.

Remark 9.3 (Status). None of this has been evaluated. These are hypotheses the framework generates rather than results it establishes, and two of them are unavailable to a governance system that does not represent evidence as a derivation structure — which is a testable claim about the framework's usefulness, recorded in Section 11 as one of the things that would falsify it.

9.3 Stratification

If machine assistance is admitted into a system that governs machines, one should ask whether the account is well-founded. It is, by stratification rather than by good intentions. Let $\mathcal{G}$ be the governing machinery — the classification map, the derivation structure, the institutions, the order — and $\mathcal{A}$ the governed population. An assistant is registered as an element of $\mathcal{A}$: it has a parameter object, a fitting morphism (configuration), a contract, a classification and evidence. No element of $\mathcal{G}$ is an element of $\mathcal{A}$, and no construction here requires a fixed point.

One dependency crosses the strata and is worth naming rather than hiding: if an assistant drafts a regime encoding, part of $\mathcal{G}$ has been produced with help from an element of $\mathcal{A}$. The resolution is that the dependency is mediated by an oracle which is itself not machine-produced — the satisfaction condition is a property of institutions, and it is evaluated mechanically. *Generation may cross the strata; acceptance may not.*

10 Related work

Categorical foundations for learning and probability. The **Para** construction and its use in gradient-based learning is due to Fong, Spivak and Tuyeras [4], elaborated by Cruttwell and collaborators [5] and situated in a wider setting by Capucci and collaborators [6]. Our use is orthogonal: we are not concerned with how parameters are optimised, but with the fact that a parameter object exists and may be inhabited by procedures other than optimisation — which is what makes Proposition 2.3 a classification of the population rather than of a technique. Markov categories are due to Fritz [1] and Cho and Jacobs [2]; we use them for one specific reason, which is that copying is explicit and Remark 4.7 needs it to be.

Order, subtyping and variance. Theorem 3.6 is elementary lattice theory [7, 8], and Definition 3.2 is the record-subtyping order with contravariant inputs [9] in the behavioural sense of [10]. We claim no novelty in the order. What appears not to have been done is to identify the four governance questions of Proposition 3.9 as that one order, and in particular to use the *meet* — rather than a pairwise compatibility check — as the answer to whether one feature set can serve several models, with the partiality of the meet (Proposition 3.8) as an informative refusal.

Feature stores and semantic layers. The engineering prior art for Section 5 is substantial and largely uncited by the governance literature. Feature stores — Feast [29], Tecton [30], Hopsworks [31] — provide point-in-time correct joins, feature versioning and online/offline consistency, and their treatment of the two clocks is essentially the one we formalise. Semantic layers — dbt [32], Cube [33], Malloy [34] — provide the definition-once, compose-many discipline that Proposition 3.11 makes algebraic. What these systems do not supply, and what this paper adds, is the operator formulation with the saturation law (Corollary 5.4): the reproducibility guarantee is usually described as a property of a correctly written pipeline rather than as a property of the read, and the difference shows up precisely in the choice of $\min(\ell, a)$ over a, which is the bug in Section 5.4. Neither family represents materiality, approved use, independent challenge or remediation, which is the other half of the division this paper's introduction describes.

Provenance. Semiring provenance is due to Green, Karvounarakis and Tannen [11, 12], with the hierarchy of [13, 14]. Existing applications are to query answering. Ours are to assurance evidence, where the derivations are governance inferences, and to derived features, where Proposition 6.3 turns a pipeline convention into a homomorphism. Our negative result (Proposition 6.4) is the elementary observation that a currency valuation is an aggregate rather than an annotation, which is the difficulty [15] addresses properly.

Bitemporal data. Snodgrass [16] is the standard reference; SQL:2011 brought system- and application-time periods into the standard [17]. Leakage as a machine-learning failure mode is documented by Kaufman and collaborators [18]. The contribution here is not the two clocks but Proposition 5.2.

Indexed families. Fibrations and the Grothendieck construction are standard [19], and Proposition 2.5 is an unremarkable instance. What we have not seen stated is Proposition 2.7: that an indexed family of *obligations* carries a constraint on its base which an indexed family of mathematical structures does not, because the totality of the family is a property somebody will rely on, and a base whose values are supplied rather than computed cannot support both totality and free extension. The constraint is invisible in the mathematics and decisive in the application, which is the general shape of what this paper is trying to do.

Specification over multiple logics, contracts, abstraction, lenses. Institution theory is due to Goguen and Burstall [20, 21]; we are not aware of prior application to overlapping normative regimes over a shared asset inventory, nor of the use of the satisfaction condition as a consistency test for regulatory encodings. Assume–guarantee contracts are due to Benveniste and collaborators [22, 23]; we import the algebra unchanged and observe that operating boundaries and performance envelopes are exactly assumptions and guarantees. Sound abstraction [24] supplies the criterion for when a summary document is trustworthy, and well-behaved lenses [25] the discipline for regenerating one; both are stated here and neither is executable in the reference implementation, which Section 8 records.

Documentation and metadata practice. Model cards [27] and datasheets [28] specify *what* a summary should contain; sound abstraction contributes a criterion for *when* a summary is trustworthy, independent of its contents. Registry and lineage systems address a subset of this territory but assume a homogeneous population of learned artefacts, which is the assumption Proposition 2.3 removes.

11 Limitations and falsification

No implementation study. There is a reference implementation and its law suite runs, which is more than a conceptual paper usually has and much less than evidence. We have not shown that a system built on this foundation is easier to construct, extend or operate than one built without it. That is the principal missing evidence.

The derivations still read declarations. Section 2.6 states this for the trainability class, and the point generalises: the order compares declared schemas, the operator reads declared clocks, the polynomial is built over declared derivation edges. The claim is that the declared surface shrinks and becomes checkable, not that it vanishes. Where the fallback direction of a derivation is permissive — as it is for an artefact whose fitting procedure was never recorded — the old failure mode returns in a smaller place.

Two variance rules for outputs. Remark 4.3 records that composition and version substitution judge output narrowing differently. Both readings are defensible; holding both is not, and this is the sort of divergence that Proposition 3.9 exists to prevent.

Five laws do not execute. They are listed in Section 8 with reasons. In particular Corollary 4.8 — the interaction premium, which we present as one of the paper's more useful consequences — is not computed anywhere. Theorem 4.6 says what a correct aggregate cannot be, and does not construct one.

The lattice is finite-fragment, and half of it has no caller. Theorem 3.6 gives no bottom element, and the meet is partial. Both are honest and both mean the structure is weaker than "schemas form a complete lattice" would suggest. Separately, the order is called from three production paths while the meet and join are called only from the law suite: the fourth line of Proposition 3.9 is a question the structure can now answer and nothing yet asks. A reader should not take the availability of an operation for evidence that it is in use.

Formalising normative text is lossy and contestable. Encoding a regime as an institution commits to a reading of prose that is deliberately open-textured. The encoding is a claim about the regulation, not the regulation, and different institutions may reasonably encode the same text differently. The formalism makes the claim explicit, citable and testable for internal consistency; it does not make it correct.

Oracles are necessary, not sufficient. Proposition 9.2 says nothing about what an unchecked output does to the humans around it, and the instruments of Section 9.2 are hypotheses we have not evaluated.

The oracle boundary may be drawn too conveniently. Observation 7.1 classifies the decisions that matter most as admitting no oracle, which is a comfortable conclusion for anyone who would prefer humans to retain them. We believe the argument is sound — a rule that constitutes a standard cannot be checked against that standard — but note that it is exactly the sort of argument whose conclusion should be scrutinised in proportion to how much one likes it.

What would falsify the account. Five things. First, an artefact class in the domain that cannot be presented as a (P, φ) pair without distortion. Second, a governance question about fit that is genuinely not an instance of $\sqsubseteq$ — which would show Proposition 3.9 to be a coincidence

of four cases rather than a unification. Third, a fragility-sensitive aggregate risk measure that is nonetheless compositional, which Theorem 4.6 forbids, so a purported example would indicate an error in our failure semantics. Fourth, and most likely, a demonstration that practitioners cannot use the derivations the formalism produces — that a refusal naming a slot with no meet is no more actionable in practice than a flag with a comment. Fifth, a finding that the review instruments of Section 9.2 do not move the fluency effect. The fourth and fifth are empirical questions about human factors and we cannot settle either from the armchair.

12 Conclusion

Model governance is treated as an engineering problem with a compliance flavour. We have argued that it is better understood as a problem with a missing foundation, and specifically that its characteristic failures are declarations standing where derivations belong.

Four of those declarations do not need to be made. What kind of artefact this is derives from how its parameter object is inhabited, which makes a closed-form pricer a type rather than an exception. Whether one thing fits where another fitted derives from a single partial order, whose meet answers a question that four separate implementations could not even ask, and whose failure to have a meet is the honest answer to whether one feature set can serve two models. What a training row could have known derives from an operator whose fourth property — saturation at the label — *is* reproducibility, and is supplied by a min that is easy to omit and whose omission is invisible. What a claim rests on derives from a polynomial, and so does the ingest clock of a derived feature, which upgrades a rule somebody could forget into a homomorphism that has no exceptions.

We have been equally concerned to say what does not derive. Aggregate risk cannot be compositional, and the obstruction is the copy map. Currency is not a semiring valuation, and no choice of constants makes it one. Five of twenty-one laws do not execute. And some facts — the tiering rule, the regime encoding, the approval — constitute the standard they would be checked against, and are therefore irreducibly declarations made by somebody accountable.

That last boundary turned out to be the useful one. Having drawn it for governance reasons, we found that it answers a question we did not set out to ask: *where may a machine do this work?* The criterion is not the model's capability nor the task's sensitivity, but whether the fact was derived or declared. Where it derives, generation is free because acceptance is checked. Where it constitutes the standard, there is nothing for the automation to be checked against, and the question is not one of risk appetite but of category.

None of the mathematics is new. What we suggest is that a domain served by tools that each solve half the problem is not waiting for a better product but for a definition — and that the definition, once written down and made executable, pays for itself twice.

References

[1] T. Fritz. A synthetic approach to Markov kernels, conditional independence and theorems on sufficient statistics. *Advances in Mathematics*, 370:107239, 2020.

[2] K. Cho, B. Jacobs. Disintegration and Bayesian inversion via string diagrams. *Mathematical Structures in Computer Science*, 29(7):938–971, 2019.

[3] M. Giry. A categorical approach to probability theory. In *Categorical Aspects of Topology and Analysis*, Lecture Notes in Mathematics 915, Springer, 1982, pp. 68–85.

[4] B. Fong, D. I. Spivak, R. Tuyeras. Backprop as functor: a compositional perspective on supervised learning. *LICS*, 2019.

[5] G. S. H. Cruttwell, B. Gavranović, N. Ghani, P. Wilson, F. Zanasi. Categorical foundations of gradient-based learning. *ESOP*, 2022.

[6] M. Capucci, B. Gavranović, J. Hedges, E. F. Rischel. Towards foundations of categorical cybernetics. *Applied Category Theory*, EPTCS, 2021.

[7] B. A. Davey, H. A. Priestley. *Introduction to Lattices and Order*. 2nd edition, Cambridge University Press, 2002.

[8] G. Birkhoff. *Lattice Theory*. 3rd edition, American Mathematical Society Colloquium Publications 25, 1967.

[9] L. Cardelli, P. Wegner. On understanding types, data abstraction, and polymorphism. *ACM Computing Surveys*, 17(4):471–523, 1985.

[10] B. H. Liskov, J. M. Wing. A behavioral notion of subtyping. *ACM Transactions on Programming Languages and Systems*, 16(6):1811–1841, 1994.

[11] T. J. Green, G. Karvounarakis, V. Tannen. Provenance semirings. *PODS*, 2007.

[12] T. J. Green, V. Tannen. The semiring framework for database provenance. *PODS*, 2017.

[13] P. Buneman, S. Khanna, W.-C. Tan. Why and where: a characterization of data provenance. *ICDT*, 2001.

[14] J. Cheney, L. Chiticariu, W.-C. Tan. Provenance in databases: why, how, and where. *Foundations and Trends in Databases*, 1(4):379–474, 2009.

[15] Y. Amsterdamer, D. Deutch, V. Tannen. Provenance for aggregate queries. *PODS*, 2011.

[16] R. T. Snodgrass. *Developing Time-Oriented Database Applications in SQL*. Morgan Kaufmann, 2000.

[17] K. Kulkarni, J.-E. Michels. Temporal features in SQL:2011. *ACM SIGMOD Record*, 41(3):34–43, 2012.

[18] S. Kaufman, S. Rosset, C. Perlich, O. Stitelman. Leakage in data mining: formulation, detection, and avoidance. *ACM Transactions on Knowledge Discovery from Data*, 6(4), 2012.

[19] B. Jacobs. *Categorical Logic and Type Theory*. Studies in Logic and the Foundations of Mathematics 141, Elsevier, 1999.

[20] J. A. Goguen, R. M. Burstall. Institutions: abstract model theory for specification and programming. *Journal of the ACM*, 39(1):95–146, 1992.

[21] R. Diaconescu. *Institution-independent Model Theory*. Birkhäuser, 2008.

[22] A. Benveniste, B. Caillaud, D. Nickovic, R. Passerone, J.-B. Raclet, P. Reinkemeier, A. Sangiovanni-Vincentelli, W. Damm, T. A. Henzinger, K. G. Larsen. Contracts for system design. *Foundations and Trends in Electronic Design Automation*, 12(2–3), 2018.

[23] I. Incer. *The Algebra of Contracts*. Ph.D. thesis, University of California, Berkeley, 2022.

[24] P. Cousot, R. Cousot. Abstract interpretation: a unified lattice model for static analysis of programs by construction or approximation of fixpoints. *POPL*, 1977.

[25] J. N. Foster, M. B. Greenwald, J. T. Moore, B. C. Pierce, A. Schmitt. Combinators for bidirectional tree transformations: a linguistic approach to the view-update problem. *ACM TOPLAS*, 29(3), 2007.

[26] K. Claessen, J. Hughes. QuickCheck: a lightweight tool for random testing of Haskell programs. *ICFP*, 2000.

[27] M. Mitchell, S. Wu, A. Zaldivar, P. Barnes, L. Vasserman, B. Hutchinson, E. Spitzer, I. D. Raji, T. Gebru. Model cards for model reporting. *FAT**, 2019.

[28] T. Gebru, J. Morgenstern, B. Vecchione, J. W. Vaughan, H. Wallach, H. Daumé III, K. Crawford. Datasheets for datasets. *Communications of the ACM*, 64(12):86–92, 2021.

[29] Feast: an open source feature store for machine learning. `https://feast.dev`.

[30] Tecton: a feature platform for machine learning. `https://www.tecton.ai`.

[31] Hopsworks feature store. `https://www.hopsworks.ai`.

[32] dbt: the analytics engineering workflow. dbt Labs. `https://www.getdbt.com`.

[33] Cube: a semantic layer for data applications. `https://cube.dev`.

[34] Malloy: an experimental language for data. `https://www.malloydata.dev`.

[35] Board of Governors of the Federal Reserve System, Federal Deposit Insurance Corporation, Office of the Comptroller of the Currency. *Supervisory Guidance on Model Risk Management*. SR 26-2 / OCC Bulletin 2026-13, 17 April 2026.

[36] Prudential Regulation Authority, Bank of England. *Model Risk Management Principles for Banks*. Supervisory Statement SS1/23, May 2023 (amended April 2026).

[37] European Parliament and Council. *Regulation (EU) 2024/1689 laying down harmonised rules on artificial intelligence*, 2024.

© 2026 Ashutosh Sinha <ajsinha@gmail.com>. Licensed under Creative Commons Attribution–NonCommercial–NoDerivatives 4.0 International (CC BY-NC-ND 4.0), `https://creativecommons.org/licenses/by-nc-nd/4.0/`. Quoted supervisory guidance, regulation and scholarly work remain the property of their respective owners. This is a research document and is not legal, regulatory or financial advice.